МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ 4

Работа со списками и кортежами в языке Python

Вариант 13

по дисциплине «Программирование на языке Python»

Выполнил
Тебаев Мансур Дахирович
Группа ЭНЭ-б-о-25-2
Проверил _____

Ставрополь 2026

1 Тема и цель работы

Тема — Работа со списками и кортежами в языке Python. Освоить создание и обход списков и кортежей, отбор и накопление, поиск первых экстремумов, сортировку копий и обработку записей.

Во всех обязательных группах индивидуальных заданий выбран вариант 13. Количество групп: 3.

Локальный проект: python-lab-04. Подготовлен локальный Git репозиторий; удалённый адрес будет добавлен после публикации владельцем. Ссылка на удалённый репозиторий: не опубликован.

2 Среда и порядок запуска

Проверка выполнена в Windows 10, Python 3.14.6, с использованием виртуального окружения. Сторонние библиотеки для учебных программ не требуются. Числа вводятся в десятичной записи с точкой; если иное не указано, поля вводятся отдельно, каждое с новой строки. Для списков вводится одна строка чисел через пробел.

Запуск и пошаговая отладка выполнены в VS Code 1.144.0 с Python Debugger 2026.6.0 через Extension Host API. Контрольные входы поданы из подготовленных данных. Отладчик подтвердил значения переменных и следующий шаг; журнал сохранён в results/vscode_debug.json.

Из папки проекта для создания собственного окружения и запуска используются команды:

```
python -m venv .venv
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe selected_sum.py
.\.venv\Scripts\python.exe verify.py
```

Файл verify.py использует тот интерпретатор, которым он запущен. Эталонные данные лежат в fixtures.json, полные результаты — в results/checks.json и results/checks.txt. Код 0 ожидается для корректного ввода, код 1 — для проверяемых ошибок диапазона или формата. Импорт учебных модулей не запускает диалог ввода благодаря защите __name__.

3 Разобранные примеры

Сумма элементов с ограничением модуля

Ввести ровно десять целых чисел. Найти сумму элементов, модуль которых меньше пяти. Решить циклом и проверить выражением генератором.

Условие строгое: −5 и 5 не включаются. Накопитель начинается с нуля; отсутствие выбранных элементов даёт сумму 0. Проверочный sum проходит по тем же данным без изменения списка.

Файл полного решения: python-lab-04/selected_sum.py.

```
import sys


def main():
    numbers = list(map(int, input().split()))
    if len(numbers) != 10:
        print('Ошибка: требуется 10 чисел', file=sys.stderr)
        return 1
    total = 0
    for number in numbers:
        if abs(number) < 5:
            total += number
    check = sum(number for number in numbers if abs(number) < 5)
    print(f'Сумма: {total}')
    print(f'Проверка: {check}')
    return 0


if __name__ == '__main__':
    sys.exit(main())
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
1 -2 3 -4 5 -5 0 9 10 4	Сумма: 2 Проверка: 2	Сумма: 2 Проверка: 2

Ввод или сценарий	Ожидается	Получено
5 6 7 8 9 10 11 12 13 14	Сумма: 0 Проверка: 0	Сумма: 0 Проверка: 0
0 0 0 0 0 0 0 0 0 0	Сумма: 0 Проверка: 0	Сумма: 0 Проверка: 0
1 2	Ошибка: требуется 10 чисел Код 1	Ошибка: требуется 10 чисел Код 1
1 -2 5 -5 4 0 10 -3 2 8	Сумма: 2 Проверка: 2	Сумма: 2 Проверка: 2
5 6 7 8 9 -5 -6 -7 -8 -9	Сумма: 0 Проверка: 0	Сумма: 0 Проверка: 0
1 1 1 1 1 1 1 1 1 1	Сумма: 10 Проверка: 10	Сумма: 10 Проверка: 10
1 2 3	Ошибка: требуется 10 чисел Код 1	Ошибка: требуется 10 чисел Код 1

Элементы между первыми экстремумами

В непустом списке целых чисел найти количество положительных элементов строго между первым минимальным и первым максимальным элементами. Нумерация индексов начинается с нуля.

Индексы меняются только при строгом улучшении значения, поэтому равный экстремум не заменяет первый. Границы упорядочиваются; range(left + 1, right) исключает оба экстремума. При соседних или совпадающих индексах ответ равен нулю.

Файл полного решения: python-lab-04/between_extrema.py.

```
import sys


def main():
    numbers = list(map(int, input().split()))
```

```
    if not numbers:
        print('Ошибка: пустой список', file=sys.stderr)
        return 1
    minimum = maximum = 0
    for i, number in enumerate(numbers):
        if number < numbers[minimum]:
            minimum = i
        if number > numbers[maximum]:
            maximum = i
    left, right = sorted((minimum, maximum))
    count = 0
    for i in range(left + 1, right):
        if numbers[i] > 0:
            count += 1
    print(f'Индекс минимума: {minimum}')
    print(f'Индекс максимума: {maximum}')
    print(f'Положительных между: {count}')
    return 0


if __name__ == '__main__':
    sys.exit(main())
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
5 1 2 5	Индекс минимума: 1 Индекс максимума: 0 Положительных между: 0	Индекс минимума: 1 Индекс максимума: 0 Положительных между: 0
1 2 3 9	Индекс минимума: 0 Индекс максимума: 3 Положительных между: 2	Индекс минимума: 0 Индекс максимума: 3 Положительных между: 2

Ввод или сценарий	Ожидается	Получено
9 -1 2 0	Индекс минимума: 1 Индекс максимума: 0 Положительных между: 0	Индекс минимума: 1 Индекс максимума: 0 Положительных между: 0
7	Индекс минимума: 0 Индекс максимума: 0 Положительных между: 0	Индекс минимума: 0 Индекс максимума: 0 Положительных между: 0
2 2 2	Индекс минимума: 0 Индекс максимума: 0 Положительных между: 0	Индекс минимума: 0 Индекс максимума: 0 Положительных между: 0
9 2 1	Индекс минимума: 2 Индекс максимума: 0 Положительных между: 1	Индекс минимума: 2 Индекс максимума: 0 Положительных между: 1
Пустой список	Ошибка: пустой список Код 1	Ошибка: пустой список Код 1
1 2 -1 3 5	Индекс минимума: 2 Индекс максимума: 4 Положительных между: 1	Индекс минимума: 2 Индекс максимума: 4 Положительных между: 1
9 2 3 -1	Индекс минимума: 3 Индекс максимума: 0 Положительных между: 2	Индекс минимума: 3 Индекс максимума: 0 Положительных между: 2

Фильтрация и сортировка копии

Удалить отрицательные элементы, отсортировать оставшиеся по убыванию и сохранить исходный список. Пустой ввод допустим.

Включение создаёт новый список, sorted — ещё один. Ноль сохраняется. Удалять элементы из исходного списка во время обхода не требуется; исходные данные выводятся для контроля.

Файл полного решения: python-lab-04/filter_sort.py.

```
def main():
    numbers = list(map(int, input().split()))
    nonnegative = [number for number in numbers if number >= 0]
    ordered = sorted(nonnegative, reverse=True)
    print('Исходный:', numbers)
    print('Неотрицательные:', nonnegative)
    print('По убыванию:', ordered)


if __name__ == '__main__':
    main()
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
3 -1 0 2	Исходный: [3, -1, 0, 2] Неотрицательные: [3, 0, 2] По убыванию: [3, 2, 0]	Исходный: [3, -1, 0, 2] Неотрицательные: [3, 0, 2] По убыванию: [3, 2, 0]
-2 -1	Исходный: [-2, -1] Неотрицательные: [] По убыванию: []	Исходный: [-2, -1] Неотрицательные: [] По убыванию: []
Пустой список	Исходный: [] Неотрицательные: [] По убыванию: []	Исходный: [] Неотрицательные: [] По убыванию: []
3 -1 0 3 -2 5	Исходный: [3, -1, 0, 3, -2, 5] Неотрицательные: [3, 0, 3, 5] По убыванию: [5, 3, 3, 0]	Исходный: [3, -1, 0, 3, -2, 5] Неотрицательные: [3, 0, 3, 5] По убыванию: [5, 3, 3, 0]
-3 -2 -1	Исходный: [-3, -2, -1] Неотрицательные: [] По убыванию: []	Исходный: [-3, -2, -1] Неотрицательные: [] По убыванию: []
0	Исходный: [0] Неотрицательные: [0] По убыванию: [0]	Исходный: [0] Неотрицательные: [0] По убыванию: [0]

Список записей кортежей

Ввести положительное число измерений, затем отдельно название датчика и конечную вещественную температуру каждого. Построить список кортежей (название, температура). Вычислить среднюю температуру и вывести записи строго выше неё по убыванию температуры.

Название может содержать пробелы. При сортировке кортеж перемещается целиком, поэтому поля сохраняют связь. Одинаковые температуры сохраняют порядок ввода благодаря устойчивости sorted. Одно измерение или все равные температуры дают пустой отбор.

Файл полного решения: python-lab-04/sensor_readings.py.

```
import sys


def main():
    n = int(input())
    if n <= 0:
        print('Ошибка: n должно быть положительным', file=sys.stderr)
        return 1
    readings = []
    for _ in range(n):
        name = input().strip()
        temperature = float(input())
        readings.append((name, temperature))
    average = sum(item[1] for item in readings) / n
    selected = [item for item in readings if item[1] > average]
    ordered = sorted(selected, key=lambda item: item[1], reverse=True)
    print(f'Средняя: {average:.2f}')
    if not ordered:
        print('Нет показаний выше средней')
    for name, temperature in ordered:
        print(f'{name}: {temperature:.2f}')
    return 0
```

```
if __name__ == '__main__':
    sys.exit(main())
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
A=20; B=24; C=22	Средняя: 22.00 B: 24.00	Средняя: 22.00 B: 24.00
Один датчик: A=7	Средняя: 7.00 Нет показаний выше средней	Средняя: 7.00 Нет показаний выше средней
B=24; A=24; C=12	Средняя: 20.00 B: 24.00 A: 24.00	Средняя: 20.00 B: 24.00 A: 24.00
0	Ошибка: n должно быть положительным Код 1	Ошибка: n должно быть положительным Код 1
T1=20; T2=24; T3=22	Средняя: 22.00 T2: 24.00	Средняя: 22.00 T2: 24.00
T1=T2=10	Средняя: 10.00 Нет показаний выше средней	Средняя: 10.00 Нет показаний выше средней
T1=−5	Средняя: -5.00 Нет показаний выше средней	Средняя: -5.00 Нет показаний выше средней
T1=0; T2=T3=6	Средняя: 4.00 T2: 6.00 T3: 6.00	Средняя: 4.00 T2: 6.00 T3: 6.00

4 Индивидуальные задания

Индивидуальное задание 1 вариант 13

Ввести ровно десять целых чисел. Найти сумму и количество элементов, модуль которых меньше трёх или которые делятся на девять без остатка.

Применено логическое or в одном условии. Ноль удовлетворяет обеим частям, но учитывается один раз. Основное решение выполнено циклом; эквивалентное включение используется для проверки суммы и количества.

Файл полного решения: python-lab-04/individual_01.py.

```
import sys


def main():
    numbers = list(map(int, input().split()))
    if len(numbers) != 10:
        print('Ошибка: требуется 10 чисел', file=sys.stderr)
        return 1
    total = count = 0
    for number in numbers:
        if abs(number) < 3 or number % 9 == 0:
            total += number
            count += 1
    selected = [x for x in numbers if abs(x) < 3 or x % 9 == 0]
    print(f'Сумма: {total}')
    print(f'Количество: {count}')
    print(f'Проверка: {sum(selected)}, {len(selected)}')
    return 0


if __name__ == '__main__':
    sys.exit(main())
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
0 1 -2 3 9 -9 18 4 5 6	Сумма: 17 Количество: 6 Проверка: 17, 6	Сумма: 17 Количество: 6 Проверка: 17, 6
3 4 5 6 7 8 10 11 12 13	Сумма: 0 Количество: 0 Проверка: 0, 0	Сумма: 0 Количество: 0 Проверка: 0, 0
0 0 0 0 0 0 0 0 0 0	Сумма: 0 Количество: 10 Проверка: 0, 10	Сумма: 0 Количество: 10 Проверка: 0, 10
-2 -1 0 1 2 9 -9 18 -18 27	Сумма: 27 Количество: 10 Проверка: 27, 10	Сумма: 27 Количество: 10 Проверка: 27, 10
-3 3 -2 2 -9 9 4 5 6 7	Сумма: 0 Количество: 4 Проверка: 0, 4	Сумма: 0 Количество: 4 Проверка: 0, 4
1	Ошибка: требуется 10 чисел Код 1	Ошибка: требуется 10 чисел Код 1

Индивидуальное задание 2 вариант 13

В непустом списке вещественных чисел определить количество нулевых элементов и сумму элементов, расположенных после первого минимального элемента. Отсортировать копию списка по возрастанию модулей элементов.

Равный минимум не заменяет ранее найденный индекс. Срез начинается с minimum + 1; если минимум последний, сумма равна нулю. sorted(numbers, key=abs) сохраняет исходный список и порядок элементов с одинаковым модулем.

Файл полного решения: python-lab-04/individual_02.py.

```
import sys
```

```
def main():
    numbers = list(map(float, input().split()))
    if not numbers:
        print('Ошибка: пустой список', file=sys.stderr)
        return 1
    zero_count = 0
    minimum = 0
    for i, number in enumerate(numbers):
        if number == 0:
            zero_count += 1
        if number < numbers[minimum]:
            minimum = i
    total = 0.0
    for number in numbers[minimum + 1:]:
        total += number
    ordered = sorted(numbers, key=abs)
    print('Исходный:', numbers)
    print(f'Нулей: {zero_count}')
    print(f'Первый минимум: {minimum}')
    print(f'Сумма после: {total:.6f}')
    print('По модулю:', ordered)
    return 0


if __name__ == '__main__':
    sys.exit(main())
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
3 0 -2 4 0	Исходный: [3.0, 0.0, -2.0, 4.0, 0.0] Нулей: 2 Первый минимум: 2 Сумма после: 4.000000 По модулю: [0.0, 0.0, -2.0, 3.0, 4.0]	Исходный: [3.0, 0.0, -2.0, 4.0, 0.0] Нулей: 2 Первый минимум: 2 Сумма после: 4.000000 По модулю: [0.0, 0.0, -2.0, 3.0, 4.0]
-2 1 -2 3	Исходный: [-2.0, 1.0, -2.0, 3.0] Нулей: 0 Первый минимум: 0 Сумма после: 2.000000 По модулю: [1.0, -2.0, -2.0, 3.0]	Исходный: [-2.0, 1.0, -2.0, 3.0] Нулей: 0 Первый минимум: 0 Сумма после: 2.000000 По модулю: [1.0, -2.0, -2.0, 3.0]
0 0 0	Исходный: [0.0, 0.0, 0.0] Нулей: 3 Первый минимум: 0 Сумма после: 0.000000 По модулю: [0.0, 0.0, 0.0]	Исходный: [0.0, 0.0, 0.0] Нулей: 3 Первый минимум: 0 Сумма после: 0.000000 По модулю: [0.0, 0.0, 0.0]
5	Исходный: [5.0] Нулей: 0 Первый минимум: 0 Сумма после: 0.000000 По модулю: [5.0]	Исходный: [5.0] Нулей: 0 Первый минимум: 0 Сумма после: 0.000000 По модулю: [5.0]
-1 1 -2 2	Исходный: [-1.0, 1.0, -2.0, 2.0] Нулей: 0 Первый минимум: 2 Сумма после: 2.000000 По модулю: [-1.0, 1.0, -2.0, 2.0]	Исходный: [-1.0, 1.0, -2.0, 2.0] Нулей: 0 Первый минимум: 2 Сумма после: 2.000000 По модулю: [-1.0, 1.0, -2.0, 2.0]
Пустой список	Ошибка: пустой список Код 1	Ошибка: пустой список Код 1

Индивидуальное задание 3 вариант 13

Ввести кортеж оценок от 2 до 5. Сформировать кортеж частот (количество двоек, троек, четвёрок, пятёрок), распаковать его в четыре переменные и вывести результаты. Пустой кортеж допустим.

tuple фиксирует последовательность оценок. count определяет частоту каждой допустимой оценки. Пустой ввод приводит к четырём нулям. Значение вне диапазона 2–5 отклоняется. Кортеж с одним элементом при выводе содержит завершающую запятую.

Файл полного решения: python-lab-04/individual_03.py.

```
import sys


def main():
    grades = tuple(map(int, input().split()))
    if any(grade not in (2, 3, 4, 5) for grade in grades):
        print('Ошибка: оценки должны быть от 2 до 5', file=sys.stderr)
        return 1
    frequencies = tuple(grades.count(grade) for grade in (2, 3, 4, 5))
    twos, threes, fours, fives = frequencies
    print('Оценки:', grades)
    print('Частоты:', frequencies)
    print(f'Двоек: {twos}; троек: {threes}; четвёрок: {fours}; пятёрок: {fives}')
    return 0


if __name__ == '__main__':
    sys.exit(main())
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
2 3 4 5 5	Оценки: (2, 3, 4, 5, 5) Частоты: (1, 1, 1, 2) Двоек: 1; троек: 1; четвёрок: 1; пятёрок: 2	Оценки: (2, 3, 4, 5, 5) Частоты: (1, 1, 1, 2) Двоек: 1; троек: 1; четвёрок: 1; пятёрок: 2
Пустой кортеж	Оценки: () Частоты: (0, 0, 0, 0) Двоек: 0; троек: 0; четвёрок: 0; пятёрок: 0	Оценки: () Частоты: (0, 0, 0, 0) Двоек: 0; троек: 0; четвёрок: 0; пятёрок: 0
5 5 5	Оценки: (5, 5, 5) Частоты: (0, 0, 0, 3) Двоек: 0; троек: 0; четвёрок: 0; пятёрок: 3	Оценки: (5, 5, 5) Частоты: (0, 0, 0, 3) Двоек: 0; троек: 0; четвёрок: 0; пятёрок: 3
2 3 2 3	Оценки: (2, 3, 2, 3) Частоты: (2, 2, 0, 0) Двоек: 2; троек: 2; четвёрок: 0; пятёрок: 0	Оценки: (2, 3, 2, 3) Частоты: (2, 2, 0, 0) Двоек: 2; троек: 2; четвёрок: 0; пятёрок: 0
4	Оценки: (4,) Частоты: (0, 0, 1, 0) Двоек: 0; троек: 0; четвёрок: 1; пятёрок: 0	Оценки: (4,) Частоты: (0, 0, 1, 0) Двоек: 0; троек: 0; четвёрок: 1; пятёрок: 0
1 5	Ошибка: оценки должны быть от 2 до 5 Код 1	Ошибка: оценки должны быть от 2 до 5 Код 1

5 Трассировка и воспроизведение в редакторе

Для [5, 1, 2, 5] начальные индексы minimum = maximum = 0. Состояния после обработки каждого элемента:

Индекс i	Значение	minimum	maximum
0	5	0	0

Индекс i	Значение	minimum	maximum
1	1	1	0
2	2	1	0
3	5	1	0

Последнее значение 5 равно первому максимуму, поэтому maximum остаётся 0. Между индексами 0 и 1 нет элементов: ответ 0. Перед проверкой максимума при i = 3 видны number = 5, minimum = 1, maximum = 0; следующий шаг не меняет maximum. Значения получены трассировкой Python.

Изменяемый список подходит для накопления записей. Кортеж фиксирует состав одной записи. Присваивание b = a не копирует список, а связывает второе имя с тем же объектом; a.copy(), срез a[:] и sorted(a) создают новые внешние списки. Копирование поверхностное: вложенные изменяемые объекты могут оставаться общими.

В VS Code дополнительно проверен список [5, −2, 3, 5]. Остановка на строке 12 between_extrema.py при i = 3 показала number = 5, minimum = 1, maximum = 0. После одного шага maximum остался равен 0. Программа вывела ноль положительных элементов между экстремумами. Все проверки verify.py также успешно выполнены запуском из VS Code. Журнал остановок и вывода сохранён в results/vscode_debug.json.

6 Вывод

Разработаны и проверены 7 учебных программ. Успешно пройдены все 49 проверок, включая предусмотренные ошибочные входы. Подтверждены граничные условия, порядок обработки данных и соответствие фактических результатов ожидаемым. Закреплены поиск первых экстремумов, безопасная фильтрация, сортировка копий и распаковка кортежей.

7 Источники

Методические материалы курса. Лабораторная работа № 4. Работа со списками и кортежами в языке Python. Общая папка: https://disk.yandex.ru/d/yT9RQ8LG9aQ_UA

Python 3.14. Стандартные типы и управление потоком выполнения: https://docs.python.org/3.14/library/stdtypes.html и https://docs.python.org/3.14/tutorial/controlflow.html